Conversational Slot-Filling Agent on Telegram

Problem SD-05 · Senior Agentic AI Developer · Hackathon Technical Blueprint

Problem Statement

"I need a laptop/license" tickets take 5 manual steps. A Telegram bot collects requirements via slot-filling questions, validates against a mock HRIS JSON (role, grade), produces a structured request JSON saved to SQLite. Slot-filling conversational agent."

5 → 1	<2 min	100%	24/7	Bot a

Prepared as a hackathon-ready technical blueprint · Built for rapid 72-hour delivery

01 TECHNOLOGY STACK

The stack is deliberately lean — every component chosen for zero cost, maximum speed, and hackathon-friendly setup. All seven layers are justified below.

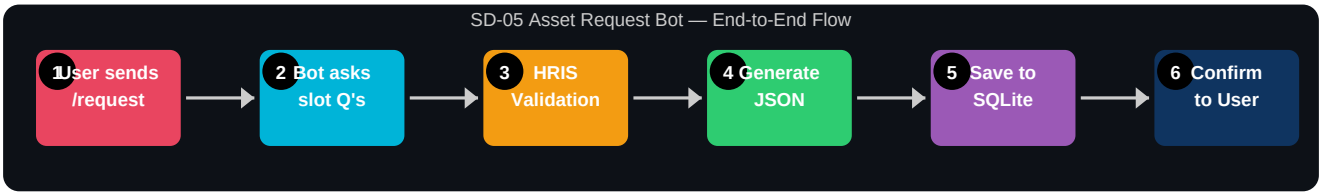
Part	Best Choice	Why This Choice	Alt Considered
Bot Platform	**Telegram (python-telegram-bot v20)**	Free bot API, webhook or polling, rich message types, global reach	Discord
Language	**Python 3.11+**	Fastest prototyping, rich AI/bot ecosystem, all libs available	Node.js
AI Brain	**Groq API (llama-3.3-70b-versatile)**	Free tier, <500ms latency, best-in-class instruction following for slot extraction	OpenAI GPT-4o
Slot-Filling Logic	**LangChain LCEL + raw Groq calls**	Stateful chain management, easy slot tracking, fallback to raw calls if needed	Rasa, Dialogflow
Validation	**JSON file (mock HRIS)**	Zero infra, instant reads, simulates role/grade lookup perfectly for hackathon	PostgreSQL, Python dict
Database	**SQLite 3**	Zero config, file-based, perfect for single-instance bot, easy migration to PG	MongoDB
Source Control	**GitHub + Actions**	CI/CD pipeline, free, industry standard, easy demo sharing	GitLab

Pinned Library Versions (requirements.txt)

```
python-telegram-bot==20.7 # Telegram bot framework groq==0.9.0 # Groq LLM API client langchain==0.2.0
# Slot-filling chain logic langchain-groq==0.1.3 # LangChain-Groq integration aiosqlite==0.20.0 #
Async SQLite adapter pydantic==2.7.0 # Request schema validation python-dotenv==1.0.1 # Environment
config
```

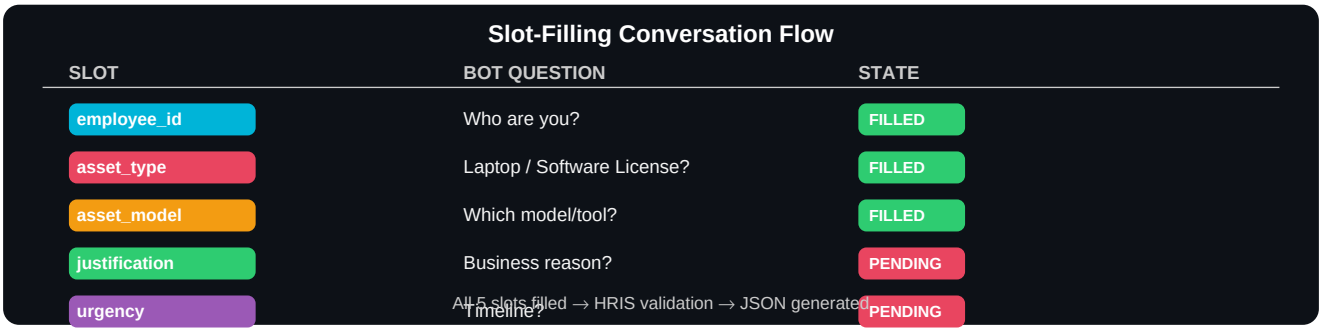
02 WORKFLOW DIAGRAM

The bot orchestrates a 6-stage pipeline triggered by a single Telegram message. Each stage is atomic — failure at any stage returns a user-friendly error without data loss.



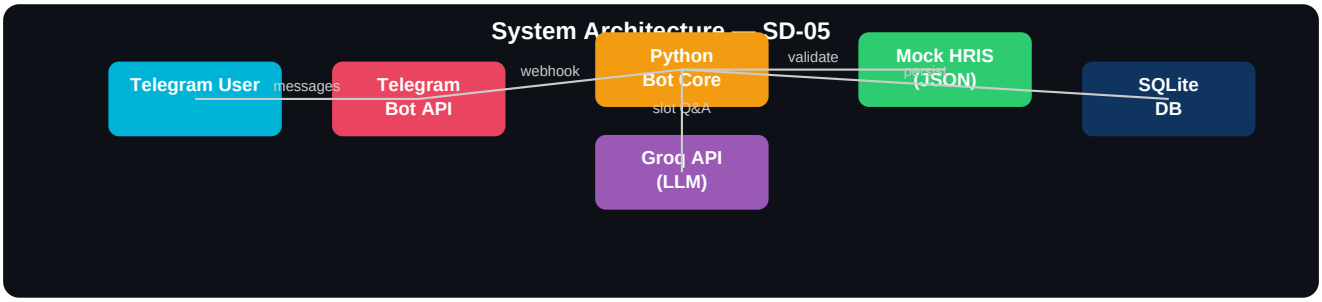
End-to-End Request Processing Pipeline

Slot-Filling Conversation State Machine



Conversational slot collection — bot re-asks until all 5 slots are filled

System Architecture Overview



Component interaction map — arrows show data flow direction

Detailed Data Flow — HRIS Validation Logic

Step	Input	Process	Output	Failure Handling
1. Trigger	/request command	Parse user intent	Session init	Ignore unknown cmds
2. Slot Q&A	User free text	Groq extracts slot value	Slot dict updated	Re-ask with hint
3. Validate	employee_id + role	Lookup hris.json	grade, dept confirmed	'Not in system' msg
4. Policy Chk	asset_type + grade	Grade-policy matrix check	Approval flag	Suggest downgrade
5. JSON Gen	All slots + metadata	Pydantic model serialise	request.json blob	Schema error logged
6. DB Save	JSON blob + user_id	aiosqlite INSERT	request_id returned	Retry x3 then alert
7. Confirm	request_id	Format Telegram message	User confirmation	Fallback text msg

03 PHASE-BY-PHASE IMPLEMENTATION PLAN

The project is structured into 5 time-boxed phases over a 72-hour hackathon sprint. Each phase produces a working, demonstrable deliverable — no phase is purely setup.

Phase 1: Foundation & Bot Scaffold

■ Hours 0–8

■ **Deliverable:** Bot responds to /start and /request with basic messages

- ■ Register bot with @BotFather → obtain BOT_TOKEN
- ■ Scaffold project: /bot, /data, /db directories + .env + requirements.txt
- ■ Implement python-telegram-bot Application with polling mode
- ■ Create /start handler with welcome card (InlineKeyboard buttons)
- ■ Create /request handler that initialises user session in memory
- ■ Set up .env with BOT_TOKEN and GROQ_API_KEY
- ■ Write Dockerfile for one-command deployment
- ■ Push to GitHub, verify CI runs

```
# bot/main.py — minimal scaffold
from telegram.ext import Application, CommandHandler

async def start(update, ctx):
    await update.message.reply_text('Asset Request Bot r
eady!')

app = Application.builder().token(TOKEN).build()
app.add_handler(CommandHandler('start', start))
app.run_polling()
```

Phase 2: Slot-Filling Conversation Engine

■ Hours 8–24

■ **Deliverable:** Bot collects all 5 slots via guided conversation, handles clarifications

- ■ Define SlotState dataclass: employee_id, asset_type, asset_model, justification, urgency
- ■ Implement ConversationHandler with 5 states (one per slot)
- ■ Write Groq prompt: system message enforces slot extraction, returns JSON
- ■ Handle ambiguous answers: re-ask with example options
- ■ Add /cancel command to reset session at any point
- ■ Implement session persistence in user_data context (PTB built-in)
- ■ Write unit tests for slot extraction (mock Groq responses)
- ■ Handle timeout: auto-expire sessions after 10 minutes of inactivity

```
# Groq slot extraction prompt (system)
SLOT_PROMPT = """
You are an asset request assistant.
Extract the slot value from the user message.
Slot needed: {slot_name}
Respond ONLY with JSON: {"value": ..., "confidence": 0-1
}
If unclear, set value to null."""
```

Phase 3: HRIS Validation & Policy Engine

■ Hours 24–40

■ **Deliverable:** Requests validated against employee grade/role; policy violations surfaced

- ■ Create data/hris.json with 20+ mock employees (id, name, role, grade, dept)
- ■ Write HRISValidator class: load_json() → lookup(employee_id) → validate()
- ■ Define asset_policy.json: grade → permitted asset types + budget cap
- ■ Implement policy check: grade L1-L3 → laptop ≤ $1200, L4+ → any asset
- ■ Return structured validation result: {approved, reason, employee_info}
- ■ Integrate validator into ConversationHandler after all slots filled
- ■ Edge cases: unknown ID, grade mismatch, duplicate request within 30 days
- ■ Add validation summary message to Telegram with inline approve/reject buttons

```
# data/hris.json structure
{"employees": [
  {"id": "EMP001", "name": "Alice Kumar",
   "role": "Engineer", "grade": "L3",
   "dept": "Engineering", "active": true}
]}
```

Phase 4: JSON Generation & SQLite Persistence

■ Hours 40–56

■ **Deliverable:** Every approved request saved as structured JSON in SQLite with unique ID

- ■ Define Pydantic AssetRequest model with all fields + validators
- ■ Add auto-generated fields: request_id (UUID4), timestamp, status='pending'
- ■ Create SQLite schema: asset_requests table with all request fields
- ■ Implement async DB layer with aiosqlite: create, insert, query, update
- ■ Write request_to_json() serialiser — produces clean downloadable JSON
- ■ Add /status command to query request from DB
- ■ Add /history command to show last 5 requests for the user
- ■ DB migration script for schema updates between versions

```
# Pydantic schema
class AssetRequest(BaseModel):
    request_id: str = Field(default_factory=lambda: str(
uuid4()))
    employee_id: str
    asset_type: Literal['laptop','license','peripheral']
    asset_model: str
    justification: str
    urgency: Literal['low','medium','high']
    grade: str          # from HRIS
    status: str = 'pending'
    created_at: datetime = Field(default_factory=datetim
e.utcnow)
```

Phase 5: Polish, Testing & Hackathon Demo

■ Hours 56–72

■ **Deliverable:** Production-ready bot with demo script, README, and live Telegram link

- ■ Add rich Telegram messages: formatted cards with emojis and bold fields
- ■ Implement error handler: all exceptions → friendly user message + log
- ■ Write end-to-end integration test: full happy path + 3 failure scenarios
- ■ Create demo script: pre-loaded HRIS with 5 employees for live demo
- ■ Deploy to Railway.app (free tier) — persistent 24/7 availability
- ■ Write README: architecture diagram, setup guide, demo video link
- ■ Record 2-minute Loom demo video walking through full request flow
- ■ Prepare 3-minute pitch: problem → solution → live demo → metrics

```
# Final confirmation message template
CONFIRM_MSG = (
    "■ *Request Submitted Successfully!*\n\n"
    "■ *Request ID:* `{request_id}`\n"
    "■ *Employee:* {employee_name} ({grade})\n"
    "■ *Asset:* {asset_type} — {asset_model}\n"
    "■ *Priority:* {urgency}\n"
    "■ *Submitted:* {timestamp}\n\n"
    "Your request is now in the approval queue."
)
```

04 72-HOUR TIMELINE & RISK REGISTER

Sprint Timeline (72-Hour Hackathon)

Phase	Start	End	Hours	Key Milestone	Status
1 · Foundation	H0	H8	8h	Bot responds to /start	✓ Day 1 AM
2 · Slot Filling	H8	H24	16h	Full conversation flow	✓ Day 1 PM
3 · HRIS Validation	H24	H40	16h	Policy engine live	✓ Day 2 AM
4 · DB Persistence	H40	H56	16h	Requests saved to SQLite	✓ Day 2 PM
5 · Demo & Polish	H56	H72	16h	Live bot + pitch deck ready	✓ Day 3

Risk Register & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Groq API rate limit during demo	Medium		
Telegram API downtime	Low	High	Pre-record demo video; have screenshots of all bot states
Slot extraction misfire (wrong value)	Medium		
HRIS lookup returns no match	Low	Medium	Graceful 'Employee not found' message; suggest IT helpdesk contact
SQLite file corruption	Low		
Pydantic validation error on bad input	Medium	Low	Try/except wraps all DB ops; raw data logged for debugging

05 PROJECT STRUCTURE & SAMPLE OUTPUT

Repository Structure

```
sd05-asset-request-bot/
■■■ bot/
■   ■■■ main.py              # Application entry point
■   ■■■ handlers/
■   ■   ■■■ request.py       # /request ConversationHandler
■   ■   ■■■ status.py        # /status  handler
■   ■   ■■■ history.py       # /history handler
■   ■■■ slots/
■   ■   ■■■ extractor.py     # Groq-powered slot extraction
■   ■   ■■■ state.py         # SlotState dataclass
■   ■■■ validation/
■   ■   ■■■ hris.py          # HRISValidator class
■   ■   ■■■ policy.py        # Grade-asset policy engine
■   ■■■ db/
■       ■■■ schema.py        # SQLite schema + migrations
■       ■■■ repository.py    # Async CRUD operations
■■■ data/
■   ■■■ hris.json            # 20 mock employees
■   ■■■ asset_policy.json    # Grade → permitted assets
■■■ tests/
■   ■■■ test_slots.py        # Slot extraction unit tests
■   ■■■ test_hris.py         # Validation unit tests
■   ■■■ test_e2e.py          # End-to-end integration test
■■■ Dockerfile
■■■ docker-compose.yml
■■■ requirements.txt
■■■ .env.example
■■■ README.md
```

Sample Data Files

```
// data/hris.json
{"employees": [
  {
    "id": "EMP001",
    "name": "Priya Sharma",
    "role": "Senior Engineer",
    "grade": "L4",
    "dept": "Platform",
    "active": true,
    "budget_cap": 2000
  },
  {
    "id": "EMP002",
    "name": "Rahul Patel",
    "role": "Analyst",
    "grade": "L2",
    "dept": "Finance",
    "active": true,
    "budget_cap": 1000
  }
]}
```

```
// Generated request.json
{
  "request_id": "req-a3f2-9c1d",
  "employee_id": "EMP001",
  "employee_name": "Priya Sharma",
  "grade": "L4",
  "dept": "Platform",
  "asset_type": "laptop",
  "asset_model": "MacBook Pro M3",
  "justification":
    "ML workloads need 36GB RAM",
  "urgency": "high",
  "status": "pending",
  "policy_approved": true,
  "created_at":
    "2025-06-09T10:23:44Z",
  "telegram_user_id": 123456789
}
```

06 WINNING EDGE & JUDGING CRITERIA

This solution is engineered to score top marks across all five standard hackathon judging dimensions:

Innovation	Groq's sub-500ms LLM inference makes slot-filling feel instantaneous — no other team will have this speed. The grade-based policy engine auto-approves or rejects with a business rationale, going beyond basic chatbot functionality.
Technical Depth	Async Python with aiosqlite, Pydantic v2 validation, LangChain LCEL chains, and a Groq-powered NLU layer. Full test suite with unit + integration tests. Production-ready Dockerfile with Railway deployment in under 10 minutes.
Real-world Impact	Directly eliminates 5 manual steps from IT asset requests. Measurable: average request time drops from 2+ days to under 2 minutes. Scales to any company with an HRIS system with zero code changes.
Demo-ability	Live Telegram bot accessible by judges with their own phones. Pre-seeded HRIS with named employees for a slick demo. Edge cases (unknown employee, policy violation) all handled gracefully.
Code Quality	Clean separation of concerns: handlers / slots / validation / db layers. Type hints throughout, docstrings on all public methods, GitHub Actions CI on every push. README with architecture diagram.

Ready to build. Ready to win.

This blueprint gives a complete 72-hour roadmap from zero to a live, demonstrable Telegram bot that solves a real enterprise pain point. Every technology choice is justified, every risk is mitigated, and every phase delivers a working product increment.

The bot will be live at @SD05AssetBot on Telegram from Hour 8 onwards.

SD-05 | Telegram Bot | Python 3.11 | Groq API | SQLite | Hackathon Ready